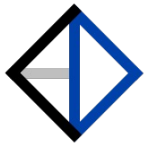
Lab 1: CUDA Basics

Computing for Data Science 2

Part A: Vector Addition

Part B: Tiled Matrix Multiplication | Part C: CUDA Tile



CODE Lab
Computing Optimization and
Data-driven Exploration Lab

Agenda

- 1. Server Environment Setup
- 2. Part A: Vector Addition
- 3. Part B: Tiled Matrix Multiplication
- 4. Part C: NVIDIA CUDA Tile Introduction

Server Access & Environment Setup

- Connect via SSH, then activate the pre-configured conda environment
- The comp2 environment includes CUDA 12.4, compatible gcc 13, and CMake 3.17

```
# 1. SSH into the server
ssh -p 22555 {username}@147.47.200.22

# 2. Activate the conda environment
conda activate comp2

# 3. Verify the prompt
(comp2) comp001@login0:~$
```

SLURM: Using Compute Nodes

- **Login node ≠ Compute node. GPU work must use srun!**
- The login node has no GPU — never compile CUDA or run GPU programs directly
- srun submits your command to a compute node with GPU access

```
# Basic format
srun -p class2 -N 1 --gres=gpu:1 <command>

# Multiple commands
srun -p class2 -N 1 --gres=gpu:1 bash -c 'cmd1 && cmd2'
```

`-p class2` → Class partition
`-N 1` → 1 node
`--gres=gpu:1` → 1 GPU

Build Workflow

- Step 1 builds the support library (CPU-only, runs on login node)
- Step 2 compiles CUDA code + generates test data (requires GPU via srun)

```
# Step 1: Build libgputk (login node OK)
cd Lab1_cuda && mkdir -p build && cd build
cmake .. -DBUILD_LIBGPUTK_LIBRARY=ON -DBUILD_LOGTIME=ON
make -j4

# Step 2: Compile + generate data (compute node)
cd ../sources
srun -p class2 -N 1 --gres=gpu:1 bash -c \
  'make all && make generators && \
   ./dataset_generator_vecadd && \
   ./dataset_generator_matmul'
```

Part A: Vector Addition

- **Goal: Implement element-wise vector addition using a CUDA kernel**
- Each GPU thread computes one element: C[i] = A[i] + B[i]
- Covers the fundamental CUDA workflow: allocate, copy, compute, copy back, free

```
A:  [1.0, 2.0, 3.0, 4.0, ...]
B:  [5.0, 6.0, 7.0, 8.0, ...]
C:  [6.0, 8.0, 10.0, 12.0, ...]  <- C[i] = A[i] + B[i]
```

CUDA Memory Model Review

- CPU and GPU have separate memory spaces; data must be explicitly copied
- This 5-step pattern is used in virtually every CUDA program

1. **`cudaMalloc`** — Allocate GPU memory
2. **`cudaMemcpy`** — Host → Device
3. **`Kernel`** — Execute on GPU
4. **`cudaMemcpy`** — Device → Host
5. **`cudaFree`** — Release GPU memory

template_vecadd.cu Structure

- **Write code only where //@@ markers appear!**
- The kernel function and main() GPU management code are your tasks

```
__global__ void vecAdd(float *in1, float *in2,
                       float *out, int len) {
  //@@ Insert kernel code here
}

int main(int argc, char **argv) {
  // ... input data loading (already implemented)

  //@@ Allocate GPU memory (cudaMalloc)
  //@@ Copy Host -> Device (cudaMemcpy)
  //@@ Set grid and block dimensions
  //@@ Launch kernel (vecAdd<<<grid, block>>>)
  //@@ Copy Device -> Host (cudaMemcpy)
  //@@ Free GPU memory (cudaFree)
}
```

Solution: Vector Addition Kernel

- Each thread computes one element using its global index
- **blockIdx.x * blockDim.x + threadIdx.x** → unique **global thread ID**
- **Boundary check** (if i < len) prevents out-of-bounds memory access

```
__global__ void vecAdd(float *in1, float *in2,
                       float *out, int len) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < len) {
    out[i] = in1[i] + in2[i];
  }
}
```

Solution: VecAdd main() — Memory Allocation

- Allocate GPU memory for both inputs and the output
- **cudaMalloc** takes a double pointer and size in bytes (not element count)
- GPU memory is not accessible from CPU — use **cudaMemcpy** to transfer

```
// Calculate total size in bytes
int size = inputLength * sizeof(float);

// Allocate GPU memory
cudaMalloc((void **)&deviceInput1, size);
cudaMalloc((void **)&deviceInput2, size);
cudaMalloc((void **)&deviceOutput, size);
```

Solution: VecAdd main() — Copy & Launch

- Copy input data to GPU, configure grid/block, and launch the kernel
- **dim3** specifies thread organization; **<<<grid, block>>>** is CUDA's launch syntax

```
// Copy input: Host -> Device
cudaMemcpy(deviceInput1, hostInput1,
           size, cudaMemcpyHostToDevice);
cudaMemcpy(deviceInput2, hostInput2,
           size, cudaMemcpyHostToDevice);

// Grid and block dimensions
int blockSize = 256;
dim3 dimBlock(blockSize);
dim3 dimGrid((inputLength + blockSize - 1)
             / blockSize);

// Launch kernel
vecAdd<<<dimGrid, dimBlock>>>(
    deviceInput1, deviceInput2,
    deviceOutput, inputLength);
```

Solution: VecAdd main() — Copy Back & Free

- After kernel execution, copy results back and release GPU memory
- Direction changes to **cudaMemcpyDeviceToHost** for the return trip
- Every **cudaMalloc** must have a matching **cudaFree** (like malloc/free)

```
// Copy result: Device -> Host
cudaMemcpy(hostOutput, deviceOutput,
           size, cudaMemcpyDeviceToHost);

// Free GPU memory
cudaFree(deviceInput1);
cudaFree(deviceInput2);
cudaFree(deviceOutput);
```

Grid/Block Dimensions Explained

- Threads are organized into blocks; blocks form a grid
- Block size is typically a multiple of 32 (warp size);
- Ceiling division ensures enough threads even when N is not a multiple of block size

```
int blockSize = 256;  // threads per block
dim3 dimBlock(blockSize);
dim3 dimGrid((inputLength + blockSize - 1)
             / blockSize);

vecAdd<<<dimGrid, dimBlock>>>(...);
```

Example: N=1000, blockSize=256

- (1000 + 255) / 256 = 4 blocks → 4 × 256 = 1024 threads
- 24 extra threads filtered out by if (i < len)

Part A: Running and Verification

- Use **srun** to run on a compute node; verify with multiple datasets
- The -t vector flag tells the verifier to expect vector-type data

```
srun -p class2 -N 1 --gres=gpu:1 \
  ./VectorAdd_template \
  -e VectorAdd/Dataset/0/output.raw \
  -i VectorAdd/Dataset/0/input0.raw,VectorAdd/Dataset/0/input1.raw \
  -t vector
```

On success: Solution is correct
Test with all datasets (0~10) to ensure correctness!

Part B: Tiled Matrix Multiplication

- **Why do we need Tiling?**
- **Naive approach:**
- Each thread reads entire row of A + column of B from global memory
- Same data accessed repeatedly by many threads → Slow!
- **Tiled approach:**
- Load small tiles into shared memory (on-chip, ~100x faster)
- Each tile is loaded once, used by all threads in the block
- Dramatically reduces global memory traffic

Tiling Concept

- Matrices are divided into TILE_WIDTH × TILE_WIDTH sub-blocks
- For each tile pair: load → sync → multiply-accumulate → sync → next tile
- TILE_WIDTH = 16 → each block has 16×16 = 256 threads

```
         B
      +----+----+
      | T0 | T1 |
      +----+----+
      | T2 | T3 |
  A   +----+----+
+----+----+
| T0 | T1 |  C[i][j] = sum(A_tile * B_tile)
+----+----+
| T2 | T3 |  1. Load tiles into shared mem
+----+----+  2. __syncthreads()
           3. Accumulate partial products
           4. __syncthreads(), next tile
```

Shared Memory Usage

- **__shared__** variables are stored in fast, on-chip memory
- Accessible by all threads in the same block (~48KB per block)
- **__syncthreads()** is required after loading to prevent data races

```
__shared__ float tileA[TILE_WIDTH][TILE_WIDTH];
__shared__ float tileB[TILE_WIDTH][TILE_WIDTH];

// All threads in this block can read/write tileA, tileB
// Must call __syncthreads() after cooperative loads
```

Memory Hierarchy Speed Comparison:

Registers → ~1 cycle (per-thread)
Shared Memory → ~5 cycles (per-block)
Global Memory → ~500 cycles (all threads)

template_matmul.cu Structure

- Same pattern as VecAdd: fill in the //@@ sections
- Kernel: declare shared memory, load tiles with boundary check, accumulate
- main(): allocate, copy, set 2D grid/block, launch, copy back, free

```
#define TILE_WIDTH 16

__global__ void matMul(float *A, float *B, float *C,
    int numARows, int numACols, int numBCols) {
  //@@ Declare shared memory for tiles
  // row, col, val computed (already provided)
  for (int t = 0; t < numTiles; t++) {
    //@@ Load A tile (boundary check)
    //@@ Load B tile (boundary check)
    //@@ __syncthreads()
    //@@ Accumulate partial product
    //@@ __syncthreads()
  }
  //@@ Write result to C (boundary check)
}
```

Solution: MatMul Kernel (1/2)

- Declare shared memory tiles and compute thread's output position
- 2D indexing: row from blockIdx.y, col from blockIdx.x

```
#define TILE_WIDTH 16

__global__ void matMul(float *A, float *B,
    float *C, int numARows, int numACols,
    int numBCols) {

  __shared__ float tileA[TILE_WIDTH][TILE_WIDTH];
  __shared__ float tileB[TILE_WIDTH][TILE_WIDTH];

  int row = blockIdx.y * TILE_WIDTH + threadIdx.y;
  int col = blockIdx.x * TILE_WIDTH + threadIdx.x;
  float val = 0.0f;

  int numTiles = (numACols + TILE_WIDTH - 1)
                 / TILE_WIDTH;
```

Solution: MatMul Kernel (2/2)

- Core loop: load tiles, sync, accumulate, sync, repeat
- Out-of-bounds positions get 0.0f (no effect on multiplication result)

```
for (int t = 0; t < numTiles; t++) {
  int aCol = t * TILE_WIDTH + threadIdx.x;
  int bRow = t * TILE_WIDTH + threadIdx.y;

  tileA[threadIdx.y][threadIdx.x] =
    (row < numARows && aCol < numACols) ? A[row * numACols + aCol] : 0.0f;
  tileB[threadIdx.y][threadIdx.x] =
    (bRow < numACols && col < numBCols) ? B[bRow * numBCols + col] : 0.0f;

  __syncthreads();

  for (int k = 0; k < TILE_WIDTH; k++)
    val += tileA[threadIdx.y][k] * tileB[k][threadIdx.x];

  __syncthreads();
}
if (row < numARows && col < numBCols)
  C[row * numBCols + col] = val;
```

Solution: MatMul main() — Alloc & Copy

- Same pattern as VecAdd, but with three separate matrix sizes
- 2D matrices stored as 1D arrays (row-major): bytes = rows × cols × sizeof(float)

```
// Calculate sizes in bytes
int sizeA = numARows * numACols * sizeof(float);
int sizeB = numBRows * numBCols * sizeof(float);
int sizeC = numCRows * numCCols * sizeof(float);

// Allocate GPU memory
cudaMalloc((void **)&deviceA, sizeA);
cudaMalloc((void **)&deviceB, sizeB);
cudaMalloc((void **)&deviceC, sizeC);

// Copy A, B to GPU
cudaMemcpy(deviceA, hostA, sizeA,
           cudaMemcpyHostToDevice);
cudaMemcpy(deviceB, hostB, sizeB,
           cudaMemcpyHostToDevice);
```

Solution: MatMul main() — Launch & Free

- Use 2D grid and block (unlike VecAdd's 1D)
- dimGrid x = columns direction, y = rows direction (note the order!)

```
// 2D grid and block dimensions
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid(
  (numCCols + TILE_WIDTH - 1) / TILE_WIDTH,
  (numCRows + TILE_WIDTH - 1) / TILE_WIDTH);

// Launch kernel
matMul<<<dimGrid, dimBlock>>>(
    deviceA, deviceB, deviceC,
    numARows, numACols, numBCols);

// Copy result back & free
cudaMemcpy(hostC, deviceC, sizeC,
           cudaMemcpyDeviceToHost);
cudaFree(deviceA);
cudaFree(deviceB);
cudaFree(deviceC);
```

Why Boundary Checks?

- **Matrix dimensions may not be multiples of TILE_WIDTH!**
- Without checks: crash or wrong results for non-aligned sizes
- Padding with 0.0f is safe because 0 × anything = 0 (no effect on sum)

```
// Example: 100x100 matrix, TILE_WIDTH=16
// 100 / 16 = 6.25 -> need 7 tiles
// Last tile: only 4 valid cols/rows, 12 padded with 0

// Load 0.0f for out-of-bounds elements
tileA[ty][tx] = (row < numARows && aCol < numACols)
    ? A[row * numACols + aCol] : 0.0f;
```

Timing Measurement (gpuTKTime)

- The template already includes timing code — results print automatically
- Record the Compute time for your report — compare across configurations

```
gpuTKTime_start(Compute,
    "Performing CUDA computation");

// ... kernel launch ...

gpuTKTime_stop(Compute,
    "Performing CUDA computation");
```

Part B: Running and Verification

- Same as Part A, but use TiledMatMul_template and -t matrix

```
srun -p class2 -N 1 --gres=gpu:1 \
  ./TiledMatMul_template \
  -e MatMul/Dataset/0/output.raw \
  -i MatMul/Dataset/0/input0.raw,MatMul/Dataset/0/input1.raw \
  -t matrix
```

Part C: What is NVIDIA CUDA Tile?

- **NVIDIA CUDA Tile (CUDA 13.1, 2025)**
- A new tile-based GPU programming model from NVIDIA
- Abstracts away SIMT thread-level details
  - focus on data chunks called **“tiles”**
- Compiler and runtime handle thread mapping automatically
- Targets Tensor Cores with forward-compatible, portable code

SIMT vs Tile Programming Model

- **SIMT (what we practiced today):**
  - You specify **each thread**'s execution: idx = threadIdx.x + blockIdx.x * blockDim.x
  - **Manual shared memory** management, __syncthreads(), boundary checks
  - Maximum control but more manual optimization required
- **Tile Model (CUDA Tile):**
  - You specify operations on data chunks (**tiles**): ct.load() → compute → ct.store()
  - No manual thread indexing, no __syncthreads(), no boundary padding
  - **Compiler handles** parallelism, memory hierarchy, Tensor Core mapping
  - Forward-compatible across GPU architectures (Ampere, Ada, Blackwell+)

CUDA Tile Ecosystem & Requirements

- **Ecosystem**
  - **CUDA Tile IR** — Virtual ISA for tile programming (MLIR-based, open source)
  - **cuTile Python** — Python DSL for writing tile kernels (pip install cuda-tile)
  - **cuTile C++** — Coming in a future CUDA release
  - **TileGym** — Example kernel library (Llama 3, DeepSeek V2 integration demos)
  - **Triton → Tile IR backend** — Bridge for existing Triton code to target Tile IR
- **Requirements**
  - GPU: Compute Capability 8.x+ (Ampere, Ada, Blackwell)
  - CUDA Toolkit 13.1+, NVIDIA Driver R580+, Python 3.10–3.13
- References
  - https://docs.nvidia.com/cuda/cutile-python/quickstart.html
  - https://github.com/NVIDIA/cutile-python
  - https://github.com/NVIDIA/cuda-tile

cuTile Python: Vector Addition

- Same VecAdd, but in cuTile Python — compare with our SIMT version!
- No thread indexing, no boundary check, no cudaMalloc/cudaMemcpy

```
import cuda.tile as ct
import cupy as cp

@ct.kernel
def vector_add(a, b, c, tile_size: ct.Constant[int]):
    pid = ct.bid(0)                 # tile (block) index
    a_tile = ct.load(a, index=(pid,), shape=(tile_size,))
    b_tile = ct.load(b, index=(pid,), shape=(tile_size,))
    result = a_tile + b_tile        # tile-level operation
    ct.store(c, index=(pid,), tile=result)

# Launch
ct.launch(stream, grid, vector_add, (a, b, c, tile_size))
```

cuTile Python: Matrix Multiplication

- ct.mma() handles tile-level multiply-accumulate
- may leverage Tensor Cores on supported HW

```
@ct.kernel
def matmul_kernel(A, B, C,
                  tm: ConstInt, tn: ConstInt, tk: ConstInt):
    bidx, bidy = ct.bid(0), ct.bid(1)
    num_k = ct.num_tiles(A, axis=1, shape=(tm, tk))
    acc = ct.full((tm, tn), 0, dtype=ct.float32)
    for k in range(num_k):
        a = ct.load(A, (bidx,k), (tm,tk))
        b = ct.load(B, (k,bidy), (tk,tn))
        acc = ct.mma(a, b, acc)
    ct.store(C, (bidx, bidy), tile=acc.astype(C.dtype))
```

Summary

- **CUDA Basic Pattern**
  - cudaMalloc → cudaMemcpy → kernel<<<>>> → cudaMemcpy → cudaFree
- **1D Indexing**
  - idx = threadIdx.x + blockIdx.x * blockDim.x
- **Shared Memory Tiling**
  - Use __shared__ arrays to reduce global memory accesses
- **__syncthreads()**
  - Synchronize twice — after load and after compute — to prevent data races and WAR hazards
- **CUDA Tile**
  - Tile-based abstraction: ct.load → compute → ct.store
  - Compiler handles threads, sync, Tensor Cores automatically